Smart Campus Academic Management System

Project Proposal

Sir Shoaib Rauf

Section

BCS-4F

Group Members

[Muhammad Ibrahim - 24K-0649]

[Muhammad ObaidUllah - 24K-0793]

[Muhammad Hasnain Siddiqui - 24K-0516]

Introduction

The **Smart Campus Academic Management System** is a database-driven system designed to manage essential academic operations within a university department. The system focuses on organizing and maintaining structured data related to **students, faculty members, courses, enrollments, attendance, and grades**.

Many universities store academic information in separate systems, which can lead to duplicated data and inconsistencies. This project demonstrates how a **single relational database** can centralize academic data and maintain consistency through **relationships, constraints, and structured queries**.

The system will be implemented using **MySQL** as the primary database. A simple interface (optional) may be created using **Python Flask or a basic web interface**, but the main focus of the project is the **database design and SQL operations**.

Main System Functions

- **Student Functions**
  - Students can:
  - View available courses
  - Register for courses
  - View their enrolled courses
  - Check attendance records
  - View grades and calculated GPA
- **Faculty Functions**
  - Faculty members can:
  - View the courses they teach
  - Record student attendance
  - Enter student grades
  - View the list of enrolled students

❖ Administrator Functions

- ➢ The administrator manages the system and can:
- ➢ Add or update students and faculty records
- ➢ Create courses
- ➢ Assign faculty members to courses
- ➢ View overall reports such as course enrollments and student performance

Users of the System

User Role	Description	Main Activities
Student	University students	Course registration, viewing grades and attendance
Faculty	Teachers responsible for courses	Record attendance and submit grades
Administrator	System manager	Manage users, courses, and system data

Technologies Used

Database (Primary Focus)

Technology	Purpose
MySQL	Main relational database
SQL	Queries, joins, and data manipulation
Stored Procedures	Automating important database operations
Views	Simplifying complex queries

MySQL Workbench	Database design and ER diagrams

Optional Interface (Secondary)

Technology	Purpose
Python + Flask	Simple backend API
HTML/CSS	Basic user interface

Scope of the System

The System WILL Include

- A relational database with **6–8 tables**
- Student, faculty, and course records
- Course enrollment management
- Attendance tracking
- Grade recording
- GPA calculation using SQL queries
- At least **3–4 complex queries using JOINs and aggregate functions**
- At least **2 stored procedures**
- An **Entity Relationship Diagram (ERD)**

The System WILL NOT Include

- Online payments
- File uploads
- Messaging or notifications
- Mobile applications
- Integration with external systems

Example Queries

Student Queries

1. View full academic transcript
2. View attendance percentage per course

3. Search available courses for registration

Faculty Queries

1. View course roster (list of enrolled students)
2. Calculate average marks for a course
3. View students with low attendance

Administrator Queries

1. Total students enrolled in each course
2. GPA distribution of students
3. List of courses taught by each faculty members

Main Database Entities

The database will contain approximately **7 core tables**:

Table	Purpose
Users	Stores login and role information
Students	Student details
Faculty	Faculty details
Courses	Course information
Enrollments	Links students with courses
Attendance	Records student attendance
Grades	Stores marks and grades

Relationships include:

- One student → many enrollments
- One course → many students

- One faculty → many courses
- One enrollment → attendance and grades

Example Stored Procedures

1. Register Student in Course

Checks if a student is already enrolled and then adds the enrollment record.

2. Calculate Student GPA

Calculates GPA based on the grades stored in the grades table.

Expected Outcome

The final system will demonstrate:

- Proper **relational schema design**
- **Primary and foreign key relationships**
- Use of **JOIN queries**
- **Aggregate SQL functions** such as SUM, AVG, and COUNT
- **Stored procedures and views**
- A clear **ER diagram**